

HIGHEST RANKED K ITEMS WITHIN A PRICE RANGE

You are given

- A grid.
- `0` = wall (cannot pass)
- `1` = empty cell
- `>1` = item whose value is also its price.

You are also given:

- starting position
- price range `[low, high]`
- integer `k`

Return & Ranking Order

Return the coordinates of the top `k` ranked items.

Items are ranked according to:

1. Shortest distance from start
2. Smaller price
3. Smaller row
4. Smaller column

We care about

Shortest distance

↓
**Immediately
BFS
comes to mind.**

During BFS

For every reachable cell, check:

```
grid[r][c] >= low && grid[r][c] <= high
```

If yes, store it.

Need to store

- distance
- price
- row
- col

because ranking depends on all four.

Data Structures

Queue

Store: `{{row, col}, distance}`

Candidate List

Store: `{distance, {price, {row, col}}}`

Later, sort.

Complexity

- **BFS:** $O(nm)$
- **Sorting:** Suppose x items satisfy price range $\rightarrow O(x \log x)$
- **Overall:** $O(nm + x \log x)$

Solution Code

```
class Solution {
public:
    vector<vector<int>> highestRankedKItems(vector<vector<int>>& grid,
    vector<int>& pricing, vector<int>& start, int k) {

        //the core idea is minimum distance chahiye to bfs
        //bfs krte rhenge aur agar budget me hua to ek vector me push krenge
        //uss vector me sirf {dist,{price,{row,col}}}
        //this vector order speciallly designed rankwise
        //baad me sort kr lenge

        int n=grid.size();
        int m=grid[0].size();

        //BFS me {{r,c},dist}

        queue<pair<pair<int,int>,int>> q;
        vector<vector<int>> seen(n,vector<int>(m,0));
        vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

        q.push({{start[0],start[1]},0});
        seen[start[0]][start[1]]=1;

        //{{dist,{price,{row,col}}}};
        vector<pair<int,pair<int,pair<int,int>>>> items;

        while(!q.empty()){
            int r=q.front().first.first;
            int c=q.front().first.second;
            int currdiss=q.front().second;
            q.pop();

            if(grid[r][c]>=pricing[0] && grid[r][c]<=pricing[1]){
                //budget me he
                items.push_back({currdiss,{grid[r][c]},{r,c}});
            }

            for(auto &it:dir){
                int nr=r+it.first;
                int nc=c+it.second;

                if(nr>=0 && nc>=0 && nr<n && nc<m){
                    if(!seen[nr][nc] && grid[nr][nc]!=0){
                        q.push({{nr,nc},currdiss+1});
                        seen[nr][nc]=1;
                    }
                }
            }
        }
    }
}
```

```

        //shopping done , ab shortlisting
        sort(items.begin(),items.end());
        vector<vector<int>> ans;
        for(int i=0;i<items.size() && k>0 ;i++){ //ya to k items daal do ya
to jitne shortlist kiye saare daalo
                                                    //agar items.size<k

        ans.push_back({items[i].second.second.first,items[i].second.second.second});
            k--;
        }

        return ans;
    }
};

```